

# 1주차 스터디 발표

# 사용자가 늘어나면, 시스템은 어떻게 바뀌어야 할까?

## 01 START

### 한 대로 시작

작은 구조로 빠르게 검증한다.

## 02 OBSERVE

### 병목을 찾고

어느 계층이 느린지 측정한다.

## 03 SCALE

### 필요한 곳을 확장

성능과 가용성을 함께 살핀다.

확장은 기술 목록이 아니라, 병목을 해결하는 의사결정이다.

# 확장의 출발점은 사용자 수보다 병목

같은 사용자 수라도 요청량·읽기/쓰기 비율·작업 비용에 따라 구조는 달라진다.

## 웹 계층

### 요청을 나눠 받기

DB 분리 → 로드밸런서  
→ 웹 서버 수평 확장  
→ 세션 외부화

## 읽기 경로

### 반복 작업 줄이기

DB 읽기 복제  
데이터 캐시  
정적 콘텐츠 CDN

## 무거운 작업·데이터

### 처리 범위 나누기

메시지 큐 + Worker  
DB 샤딩  
독립적인 확장

관찰 → 병목 식별 → 필요한 구조 도입 → 다시 측정

# 웹 서버와 DB를 먼저 분리한다

한 서버에 몰려 있던 책임을 나누면, 자원을 독립적으로 조정할 수 있다.

## BEFORE

### 단일 서버

사용자 → [ 웹 + DB ]

CPU·메모리·디스크를 공유  
서버 장애 시 전체 서비스 영향

## AFTER

### 계층 분리

사용자 → 웹 서버 → DB 서버

웹과 DB를 독립적으로 확장  
네트워크·운영 복잡성은 증가

프론트·백엔드·DB를 각각 배포했다면 이미 계층이 분리된 구조다.



# 더 강한 서버 vs 더 많은 서버

수직 확장은 단순하고, 수평 확장은 분산을 위한 설계가 필요하다.

## SCALE UP

### 한 대의 성능을 높인다 - 수직적 확장

CPU·RAM 증설  
구조가 단순하고 시작하기 쉽다.

하드웨어 한계와 단일 장애 위험

## SCALE OUT

### 서버를 추가한다 - 수평적 확장

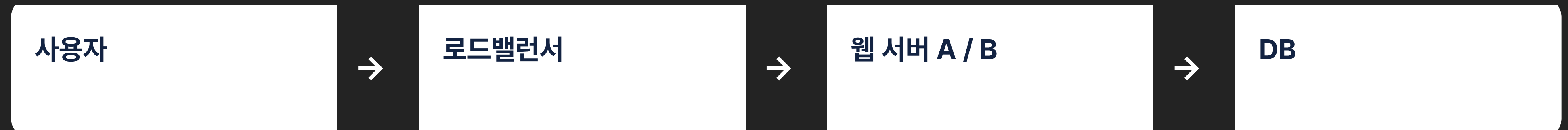
여러 서버로 요청 분산  
처리량과 장애 대응 여지를 늘린다.

로드밸런싱·상태 관리 비용 증가

수평 확장도 무한하지 않다. 공유 DB와 조정 비용이 다음 병목이 된다.

# 로드밸런서가 요청의 입구를 맡는다

여러 웹 서버로 요청을 분산하고, 상태가 나쁜 서버를 대상에서 제외한다.



## 분산

### 요청을 나눠 처리

각 웹 서버의 부담을 줄인다.  
서버 추가로 처리 용량을 늘린다.

## 장애 대응

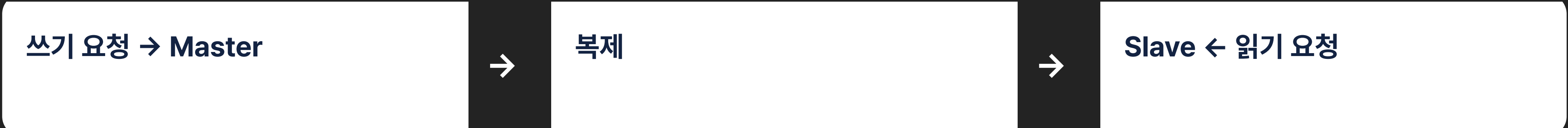
### 정상 서버로 전달

헬스 체크로 장애를 감지한다.  
로드밸런서 자체도 가용성 설계가 필요하다.

Auto Scaling은 설정한 정책과 템플릿에 따라 서버 수를 조절한다.

# 복제는 읽기를 나누고, 사본을 유지한다

Master에 쓰고 Slave에서 읽는 구조로 읽기 부하를 분산한다.



이점

## 읽기 처리량 확대

여러 사본으로 조회를 분산한다.  
장애 시 사본을 활용할 수 있다.

트레이드오프

## 복제 지연과 전환

쓰기 직후 Slave에는  
이전 값이 보일 수 있다.  
장애 전환 정책도 필요하다.

# 데이터 캐시와 CDN은 어디를 줄이는가?

반복 조회 비용과 콘텐츠 전달 거리를 각각 줄인다.

## DATA CACHE

### DB 조회를 줄인다

웹 서버 → 캐시 → (미스 시) DB

자주 조회하는 결과를 재사용

TTL·무효화·축출 정책 필요

캐시 장애 시 DB 부하 급증에 대비

## CDN

### 사용자와의 거리를 줄인다

사용자 → 가까운 에지 → 원본

이미지·영상·JS·CSS 등을 캐싱

캐시 미스 시 원본에서 가져옴

콘텐츠 갱신과 만료 정책 필요

캐시의 핵심은 저장 매체보다 '무엇을, 얼마나 오래, 어떻게 갱신할까'다.



# 어느 웹 서버가 받아도 처리할 수 있게

로그인 세션이 특정 서버에만 있으면, 요청을 자유롭게 분산하기 어렵다.

## STATEFUL

### 서버 A에 세션 저장

A로 요청 → 로그인 상태 확인  
B로 요청 → 세션이 없을 수 있음

서버 고정과 장애 대응이 복잡해진다.

## STATELESS WEB TIER

### 세션을 외부 저장소로

A / B → 공용 세션 저장소 조회  
어느 서버에서도 동일 상태에 접근

웹 서버 증설·교체가 쉬워진다.

JWT도 선택지다. 만료·폐기·갱신 정책은 별도로 설계해야 한다.

# 오래 걸리는 일은 Worker에게 넘긴다

사용자 요청 처리와 무거운 작업의 실행을 분리한다.

## PRODUCER

### 웹 서버

요청 검증 → 작업 등록  
접수 결과를 먼저 응답

## BUFFER

### 메시지 큐

작업을 보관하고 전달  
일시적인 트래픽 증가 흡수

## CONSUMER

### Worker

이메일 발송·영상 변환  
완료 상태와 결과 저장

웹 서버 → 메시지 큐 → Worker → 결과 저장

재시도·중복 처리·실패 복구가 필요하다. 외부 API 호출만으로 큐가 생기지는 않는다.

# 한 DB가 감당하기 어렵다면, 데이터를 나눈다

샤딩 키를 기준으로 데이터와 요청을 여러 DB에 분산한다.

SHARD 0

**$\text{user\_id} \% 3 = 0$**

사용자 3, 6, 9 ...

해당 사용자의 데이터를 저장

SHARD 1

**$\text{user\_id} \% 3 = 1$**

사용자 1, 4, 7 ...

해당 사용자의 데이터를 저장

SHARD 2

**$\text{user\_id} \% 3 = 2$**

사용자 2, 5, 8 ...

해당 사용자의 데이터를 저장

키 선정 → 부하 편중 방지   /   샤드 추가 → 재분배   /   샤드 간 JOIN → 복잡성

# 증상에 맞는 도구를 선택한다

하나의 기술이 모든 병목을 해결하지는 않는다.

웹 요청 처리량이 부족하다

→ 로드밸런서 + 무상태 웹 서버 확장

같은 조회가 반복된다

→ 데이터 캐시 · 정적 콘텐츠는 CDN

DB 읽기 부하가 크다

→ 읽기 Replica · 복제 지연 고려

작업이 오래 걸린다

→ 메시지 큐 + Worker

단일 DB의 용량·처리량이 한계다

→ 샤딩 · 분산 운영 비용 고려

처리량·지연 시간·오류율·자원 사용량을 보고, 바뀐 병목을 다시 확인한다.



# 설계의 이유를 설명할 수 있는가?

개념을 외우는 데서 끝내지 않고, 선택과 비용을 함께 이야기해 보자.

Q1

## 로그인이 풀린다면?

웹 서버를 1대에서 2대로 늘린 뒤  
간헐적으로 로그인 상태를 잃는다.  
무엇을 먼저 확인할까?

Q2

## 수정한 값이 안 보인다면?

쓰기 직후 조회했는데  
이전 값이 보인다.  
복제와 캐시 중 어디를 볼까?

Q3

## 샤딩이 꼭 필요할까?

DB가 느려졌다.  
바로 샤딩하기 전에  
어떤 지표와 쿼리를 볼까?

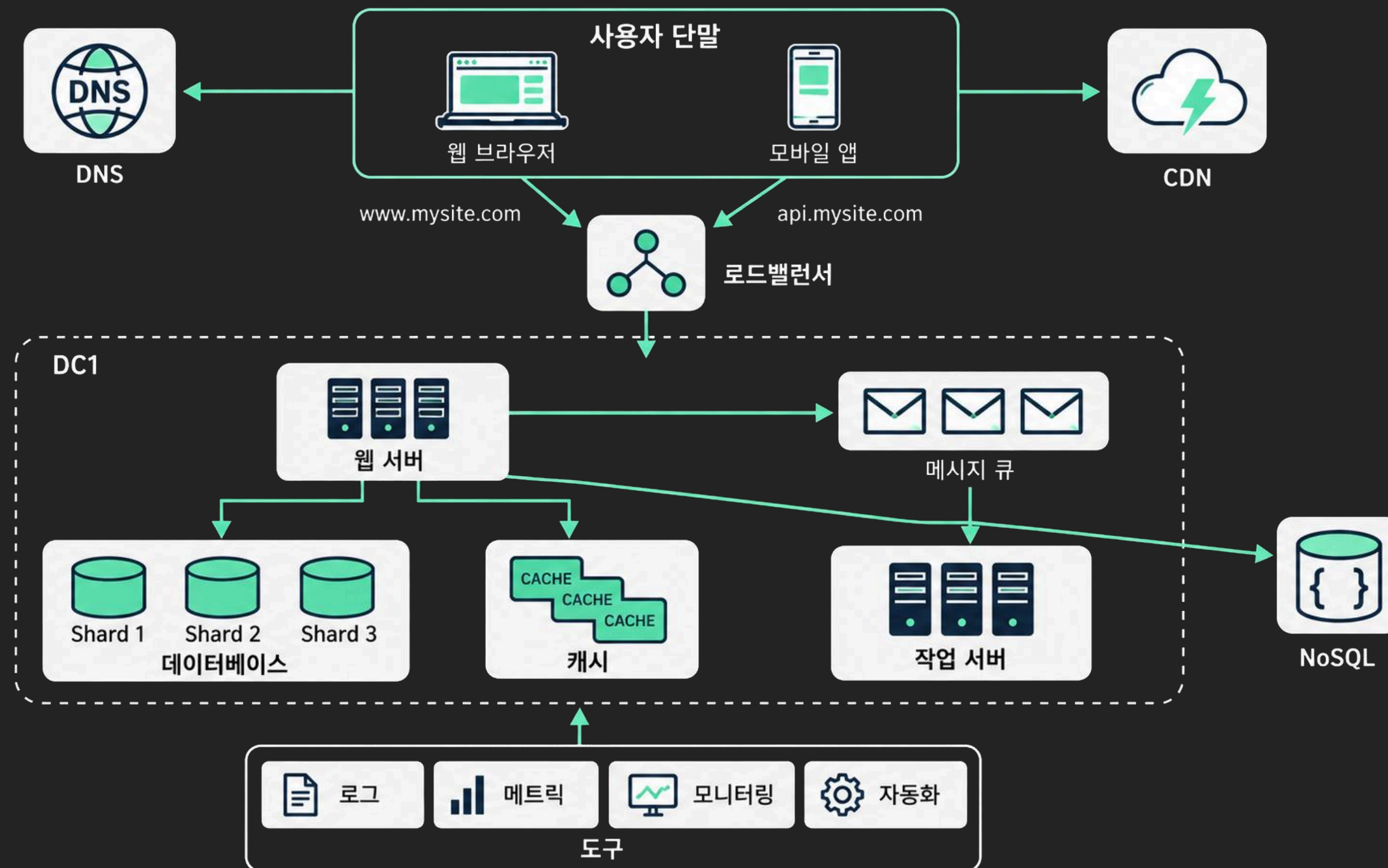
# 우리가 구현했던 서버에는 어느 기술이 적용되어 있었을까?

나는 AWS EC2 + RDS 기반으로 백엔드와 프론트 서버를 각자 배포했다.

로그인 방식으로는 JWT 토큰 인증을 선택했다.

핵심 기능은 OpenAI API 키 호출을 통해서 사용자의 요청을 우리가 설정한 프롬프트를 통해 바꿔 전달해주는 것이다.

배포는 깃허브 CI/CD 자동화를 사용했다.



DNS : 가비아를 활용한 public IP 연결

계층 분리 : EC2 + RDS를 통한 DB 분리

작업 서버 분리 : SpringBoot 서버와 작업 서버(OpenAI) 분리

자동화 : 깃허브 CI/CD 자동화

무상태 계층 : JWT토큰 인증을 사용해 웹 계층에서 상태 정보를 제거